

Super Admin Production Setup Guide

Digitex School Management System

Version: Laravel 11 | **Document date:** September 3, 2026

E-Digitex SMS — Multi-Institution Education Platform
Confidential — For authorized users only

E-Digitex Super Admin Production Setup Guide

Production site: `https://e-digitex.com`

Audience: Platform Super Admin and deployment administrator

Purpose: Prepare the platform, communication services, security, and default SMS templates before creating the first real school.

> Production must use a separate database from staging. Never run `DatabaseSeeder`, `BulkDummyDataSeeder`, `migrate:fresh`, or `migrate:fresh --seed` on the live database.

1. Required deployment state

Complete these checks before signing in:

- The main domain document root points to the Laravel `public` directory.
- HTTPS is active for `e-digitex.com`.
- The server is running the latest reviewed `main` branch.
- Production and staging use separate databases and separate `.env` files.
- The production `.env` is not committed to Git and is readable only by the server account.
- `storage` and `bootstrap/cache` are writable by PHP.
- The production database starts empty unless approved real data is being migrated.

Minimum production environment

Use real values on the server; do not copy the placeholders below literally.

```
APP_NAME="E-Digitex"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://e-digitex.com
APP_TIMEZONE=Africa/Kinshasa
APP_LOCALE=fr
APP_FALLBACK_LOCALE=en

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=YOUR_PRODUCTION_DATABASE
DB_USERNAME=YOUR_PRODUCTION_USER
DB_PASSWORD=YOUR_STRONG_DATABASE_PASSWORD

SESSION_DRIVER=database
SESSION_SECURE_COOKIE=true
SESSION_HTTP_ONLY=true
SESSION_SAME_SITE=lax
QUEUE_CONNECTION=database
CACHE_STORE=database
FILESYSTEM_DISK=local

MAIL_MAILER=smtp
MAIL_HOST=YOUR_MAIL_HOST
MAIL_PORT=465
MAIL_USERNAME=YOUR_MAIL_ACCOUNT
MAIL_PASSWORD=YOUR_MAIL_PASSWORD
MAIL_ENCRYPTION=ssl
MAIL_FROM_ADDRESS=info@e-digitex.com
MAIL_FROM_NAME="${APP_NAME}"

PLATFORM_ADMIN_EMAIL=admin@e-digitex.com
PLATFORM_ADMIN_USERNAME=digitex-admin
PLATFORM_ADMIN_PASSWORD=YOUR_ONE_TIME_STRONG_PASSWORD

REGISTRATION_ENABLED=false
PAYMENT_GATEWAY_ENV=production
LOG_LEVEL=error
```

Generate `APP_KEY` only when it is empty:

```
php artisan key:generate
```

Do not regenerate `APP_KEY` after encrypted credentials or live data exist. Doing so can make encrypted values unreadable.

First production initialization

From the application root:

```
composer install --no-dev --optimize-autoloader
php artisan storage:link
php artisan migrate --force
php artisan db:seed --class=ProductionDatabaseSeeder --force
php artisan permission:cache-reset
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

`ProductionDatabaseSeeder` installs only:

- roles, permissions, and modules;
- location reference data;
- global SMS templates;
- the configured Platform Super Admin.

It does not install demo schools, students, staff, media, invoices, or test accounts.

> Run `ProductionDatabaseSeeder` only during the first installation on a fresh database. It includes `LocationSeeder`, which rebuilds location tables and must not be rerun after real schools reference those locations. For routine releases, run migrations and `RolePermissionSeeder` separately.

2. Sign in and remain in Global View

- Open <https://e-digitex.com/login>.
- Sign in with the Platform Super Admin account.
- Immediately replace any temporary deployment password.
- Click the building icon in the top header.

- Select **Global View**.
- Confirm the header says **Global View**, not a school name.

Global View is essential. Changes made while a school is selected may apply only to that school.

3. Verify roles, permissions, and modules

Open **Permissions & Modules → Roles**.

- Confirm the standard platform and school roles exist.
- Confirm **Super Admin** has the required platform permissions.
- Confirm **School Admin** has the expected school-management permissions.
- Do not assign a school-scoped role to the Platform Super Admin.
- Reset permission cache after changing role permissions:

```
php artisan permission:cache-reset
```

Do not create real schools until the role list and menu permissions are correct; each school receives institution-scoped access derived from this platform configuration.

4. Configure packages and subscription rules

Open **Institution Management → Packages**.

Before creating schools:

- Remove or deactivate demo-only packages.
- Create the real commercial packages, for example Basic, Standard, and Premium.
- Set each package's duration, price, student limit, and staff limit.
- Select the modules included in each package.
- Decide whether a default package is assigned automatically.
- Confirm package prices and currency are production values.

Review these module groups carefully:

- students and enrollments;
- academics, sessions, grades, classes, and subjects;
- attendance;
- finance, invoices, and payments;
- examinations, marks, and report cards;
- communication and SMS templates;
- discipline, infirmary, transport, and voting;
- backups, AI, chatbot, or voice features if sold.

After a school is created, verify its subscription is active and its package includes every module the school was promised.

5. Configure platform currency

In Global View, open **Configuration → Currency**.

- Select the platform's primary currency.
- Configure CDF/USD or other secondary currency behavior if required.
- Set and verify the exchange rate.
- Confirm number format and symbol display.
- Recheck package prices after changing currency.

Each school can later receive its own financial configuration, but the platform defaults should be correct first.

6. Configure platform email (SMTP)

In Global View, open **Configuration → System Configuration → SMTP**.

Enter:

- SMTP host;
- port;
- encryption (`TLS` or `SSL`);
- username and password;

- From Address;
- From Name.

Save, then use **Send Test Email** with an inbox you can inspect.

Do not continue until:

- the test email is delivered;
 - SPF, DKIM, and DMARC are configured for the sending domain;
 - password-reset links use `https://e-digitex.com`;
 - the From Address is approved by the mail provider.
-

7. Configure global SMS and WhatsApp providers

Remain in **Global View**, then open **Configuration → ID Sender SMS**.

Provider availability

Under **Provider Availability Control**, enable only providers that schools are permitted to select. Do not expose providers that have not been tested or contracted.

System defaults

- Choose the **System Default SMS Provider**.
- Choose the **System Default WhatsApp Provider** if WhatsApp is offered.
- Enter the platform API credentials in the matching provider panel.
- Configure the approved SMS Sender ID.
- Save changes.

Schools using **System Default (Digitex Credits)** use these Global View credentials and consume school credits. Schools selecting their own provider use their own credentials.

Sender ID checklist

- The Sender ID is registered and approved by the SMS provider.
- Capitalization matches the provider approval.
- The Sender ID length satisfies the provider/country rules.

- The provider account is funded and allowed to send to target countries.
- Test numbers use international E.164 format, for example `+243...`.

Create the Infobip WhatsApp Template Name

WhatsApp allows free-form text only during the 24-hour customer-care window after the parent has messaged the business. Outside that window, E-Digitex retries with an approved Infobip/Meta template.

A. Register the template in Infobip

- Sign in to the Infobip web portal.
- Open **Channels and Numbers → Channels**.
- Select **WhatsApp → Senders**.
- Find the registered production sender used by E-Digitex.
- Open its three-dot menu and select **Manage templates**.
- Select **Register template**.
- Choose **Utility** for operational school notifications. Meta may reclassify a template based on its actual content.
- Enter a template name, for example:

```
school_notification
```

The name may contain only lowercase letters, numbers, and underscores. Do not use spaces, uppercase letters, hyphens, or accents.

- Select the language, for example **English (en)** or **French (fr)**.
- Create a text body containing exactly one body placeholder, `{{1}}`.

Recommended English body:

```
School notification: {{1}} Thank you.
```

Recommended French body:

```
Notification scolaire : {{1}} Merci.
```

- Provide a realistic sample value for `{{1}}` when Infobip requests placeholder examples.
- Submit the template for Meta approval.
- Wait until its status is **Approved**. Pending or rejected templates cannot be used for business-initiated messages.

Use a production sender that belongs to your Infobip account. A shared test sender cannot be used to register your own production template.

B. Template structure required by E-Digitex

The current E-Digitex Infobip integration supplies:

- one template name;
- one language code;
- exactly one body placeholder;
- the complete rendered school message as the value of `{{1}}`.

Therefore, do not add:

- a second body placeholder such as `{{2}}`;
- variable header fields;
- media headers;
- dynamic buttons or URL parameters;
- required button payloads.

Those fields are not supplied by the current integration and would cause template sends to fail. Any static text placed before or after `{{1}}` is displayed on every out-of-session message.

C. Connect the approved name to E-Digitex

In E-Digitex:

- Switch to **Global View** for the platform provider, or select a school when configuring that school's own Infobip provider/template.
- Open **Configuration → ID Sender SMS → Infobip**.
- Enter only the Infobip API subdomain, not the full URL. For `https://abc123.api.infobip.com`, enter `abc123`.

- Enter the API key.
- Enter **WhatsApp From Number** as the exact registered sender, digits only with country code, for example 243... .
- Enter **WhatsApp Template Name** exactly as approved, for example `school_notification`.
- Enter **Template Language** exactly as registered, for example `en` or `fr`.
- Save changes.
- Open **Test Notifications** and send to a number that has not contacted the sender during the last 24 hours. This verifies the approved-template fallback instead of only testing free-form session messaging.
- Confirm delivery and check **Message Logs** if it fails.

The Infobip template name is not the E-Digitex SMS event key. For example, `payment_received` chooses the E-Digitex message wording, while `school_notification` is the approved Infobip wrapper used to deliver that wording outside the 24-hour window.

D. Troubleshooting

- **Template not found:** Template name, language, or sender does not exactly match the approved Infobip record.
- **Template pending/rejected:** Review its status and rejection reason in Infobip; edit and resubmit it.
- **Wrong language:** Use the language code registered for that template, not merely the dashboard language.
- **Free-form works but proactive send fails:** The recipient was previously inside the 24-hour window; verify that the template is configured and approved.
- **Unexpected repeated text:** The extra text is part of the Infobip template wrapper around `{{1}}`; edit the template in Infobip and obtain approval again.
- **Invalid placeholder error:** Keep exactly one body placeholder and remove unsupported header/button parameters.
- **Wrong sender:** The template must be approved for the same WhatsApp sender number entered in E-Digitex.

Current Infobip reference: <https://www.infobip.com/docs/whatsapp/message-types-and-templates/message-templates>

Test provider delivery

Open **Configuration** → **Test Notifications**.

- Send one test SMS to a controlled phone.
- Send one test WhatsApp message if enabled.
- Confirm actual delivery, not only an API success response.
- Check **Configuration** → **Message Logs** for provider, recipient, status, and error.

Do not place API keys, tokens, or passwords inside an SMS template.

Remove exposed fallback credentials

Before production, inspect `config/sms.php`. Provider usernames, passwords, app secrets, endpoints, and test sender numbers must not be hard-coded as fallback values. Configure them through protected environment variables or encrypted Global View settings.

Any credential that has appeared in source control, documentation, screenshots, chat, or a shared `.env` must be treated as compromised and rotated at the provider. Removing it from the latest Git commit does not remove it from Git history.

8. Install and manage SMS templates

Install the default template catalogue

On a fresh production database, `ProductionDatabaseSeeder` installs the catalogue. To synchronize the current global defaults separately:

```
php artisan db:seed --class=SmsTemplateSeeder --force
```

This command uses `updateOrCreate` for global templates. It does not create dummy schools or users.

> Caution: rerunning `SmsTemplateSeeder` updates global template bodies to the versions defined in code. Export or record deliberate Global View wording changes before reseeding.

Edit a global template in the user interface

- Switch to **Global View**.

- Open **Configuration** → **SMS Templates**.
- Find the event, such as **Payment Received**.
- Click **Edit**.
- Edit the message body.
- Insert variables using the tag buttons shown in the modal.
- Keep **Active** enabled only if the event should be available.
- Review the character and SMS segment counters.
- Click **Save Changes**.

Example:

```
Dear $ParentName, we received $Amount for $StudentName.
Balance: $Balance. Thank you, $SchoolName.
```

Only the tags displayed for that event are valid. Unknown tags are rejected. Tag names are case-sensitive.

Create a new template event

The current interface edits existing event templates; it does not define a new application event from nothing. A developer must connect a new event to the code:

- Add the event key and allowed tags to `app/Services/TemplateVariableRegistry.php`.
- Add the global default to `database/seeder/SmsTemplateSeeder.php`.
- Add the notification trigger that supplies values for every tag.
- Add notification preference support if users can enable or disable its channels.
- Deploy and run:

```
php artisan db:seed --class=SmsTemplateSeeder --force
php artisan config:clear
```

- Return to **Global View** → **Configuration** → **SMS Templates** and edit/test the new event.

Changing only the database or inventing an event key in the UI does not make the application send that event.

School-specific wording

After a school exists:

- Select that school using the building icon.
- Open **Configuration** → **SMS Templates**.
- Edit an event and save it.

The saved row becomes that school's override. Other schools continue using their own override or the Global View default.

Recommended templates to review before the first school

Review at least:

- Institution Created;
- Student Welcome;
- Staff Welcome;
- Teacher Welcome;
- Guardian Welcome;
- Payment Received;
- Invoice Generated;
- Smart Fee Reminder;
- Student Arrival;
- Student Departure;
- Student Absence Alert;
- Admission and pre-enrollment messages;
- Re-enrollment invitation and reminders;
- Exam reminder and results publication;
- Homework published;
- disciplinary and parent-teacher meeting notices;
- OTP login.

Template quality rules

- Use the recipient's language and a professional tone.
 - Include `$SchoolName` where the sender may be unclear.
 - Avoid unnecessary accents or emoji when they increase SMS encoding/segments.
 - Keep normal SMS messages near 160 GSM characters when possible.
 - Never include API keys, passwords, card data, or medical details.
 - Welcome messages may contain a temporary password only if the operational policy requires it; force or instruct a password change.
 - Ensure legal consent exists for bulk or marketing messages.
 - Use the exact available tags shown by the template editor.
-

9. Configure notification channel policy

Open **Configuration → Notification Settings** in Global View.

Decide which platform events may use:

- SMS;
- WhatsApp;
- email;
- in-app bell.

Start conservatively to prevent accidental charges:

- keep expensive channels off for nonessential events;
- enable welcome and security messages only after their templates are verified;
- enable finance and attendance alerts according to the service package;
- keep in-app notifications enabled where appropriate.

After each school is created, review its notification settings because schools may need different policies.

10. Configure platform integrations and secrets

Before schools are onboarded, decide which optional services are live:

Payments

- Set production mode only after provider KYC and credentials are approved.
- Configure webhook secrets.
- Register production callback/webhook URLs under `https://e-digitex.com`.
- Perform a low-value controlled transaction.
- Confirm payment status and ledger behavior.

Hardware attendance

- Generate a strong `HARDWARE_SECRET`.
- Keep the allowed institution list empty until real institution IDs exist.
- Add institution IDs only after device ownership is verified.
- Use HTTPS for every hardware API request.

Chatbot and WhatsApp webhooks

- Configure a strong `CHATBOT_WEBHOOK_SECRET`.
- Keep webhook signature verification enabled.
- Never enable skip-verification flags in production.
- Register provider webhooks using the production domain.

AI

- Use a production API key with spending limits.
- Restrict access through packages/modules.
- Monitor usage before enabling it for schools.

Backups

- Configure server/database backups before storing school data.
- Test a restore into a non-production database.

- Configure the platform Google OAuth app if schools will use Drive backups.
 - Never connect staging and production to the same backup destination without clear separation.
-

11. Configure queue, scheduler, and monitoring

The queue worker is required for reliable asynchronous communication.

Preferred Supervisor command:

```
php artisan queue:work --sleep=3 --tries=3 --timeout=120
```

After every deployment:

```
php artisan queue:restart
```

Configure the Laravel scheduler:

```
* * * * * cd /home/ACCOUNT/APPLICATION && php artisan schedule:run >> /dev/null 2>&1
```

If Supervisor is unavailable, use a hosting-supported worker or a carefully configured `--stop-when-empty` cron.

Verify:

- queued test notifications are processed;
 - failed jobs are monitored;
 - `storage/logs/laravel.log` is writable and rotated;
 - disk and database backups are running;
 - server timezone and application timezone are intentional.
-

12. Security lock before creating schools

Complete all items:

- `APP_ENV=production`
- `APP_DEBUG=false`

- HTTPS forced and valid
- public registration disabled
- `.env` outside the public document root and permissions restricted
- no dummy institutions, users, students, media, or transactions
- no `@yopmail.com` production administrator
- all secrets are unique to production
- secrets previously sent through insecure channels have been rotated
- SMS configuration contains no hard-coded live credential fallbacks
- database user is restricted to the production database
- webhook signature verification is enabled
- payment gateways are intentionally sandbox or production
- file upload and backup storage have sufficient quota
- the Super Admin password is unique and stored securely
- GitHub branch protection is enabled for `main`
- deployment access is limited to authorized operators

Never run these commands on production:

```
php artisan migrate:fresh
php artisan migrate:fresh --seed
php artisan db:seed
php artisan db:seed --class=BulkDummyDataSeeder
```

13. Production readiness test

Before creating the first school, verify:

- Super Admin login and Global View.
- Roles and permissions.
- Package/module definitions.
- Platform currency.
- SMTP save and real email delivery.
- SMS provider save and real SMS delivery.

- WhatsApp delivery if offered.
- Message Logs record correct statuses.
- Global SMS templates exist and use valid tags.
- Queue worker processes jobs.
- Scheduler runs.
- Database and uploaded-file backups complete.
- A restore test succeeds outside production.
- No dummy records or media exist.
- Logs expose no credentials or debug traces.

Only then create the first real school.

14. First real school onboarding order

For each school:

- Create the institution with its real name, type, responsible person, phone, email, address, and official code.
- Assign the correct package and active subscription.
- Select the school from the building icon.
- Upload its real logo and branding.
- Create and activate the academic session.
- Configure school year dates, hours, late margin, and attendance behavior.
- Configure currency and exchange rate.
- Confirm enabled modules.
- Confirm the School Admin account and permissions.
- Choose System Default messaging with credits, or enter the school's own provider credentials.
- Review school-specific SMS template overrides.
- Configure notification channels.
- Create grades/classes/subjects before importing real students.

- Test one staff account, one student/guardian account, one invoice, one attendance entry, and one report card.
- Confirm school backup scheduling.

Do not bulk-import real data until this controlled end-to-end test passes.

15. Safe update procedure after go-live

```
cd /home/ACCOUNT/APPLICATION
php artisan down
git pull origin main
composer install --no-dev --optimize-autoloader
php artisan optimize:clear
php artisan migrate --force
php artisan db:seed --class=RolePermissionSeeder --force
php artisan permission:cache-reset
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan queue:restart
php artisan up
```

Run `SmsTemplateSeeder` during an update only when the release intentionally changes the global template catalogue.

Do not run `ProductionDatabaseSeeder` during routine updates after schools exist.

Run deployment steps explicitly and check every exit code. Do not consider a release successful if a migration, seeder, or cache command reports an error.

Maintain these rules:

- develop on feature branches;
- test on staging with staging-only dummy data;
- merge reviewed changes into protected `main`;
- production pulls only `main`;
- never edit application code directly on the live server;
- back up before migrations;

- record deploy time and commit hash;
 - verify login, queues, logs, and a critical school workflow after each deploy.
-

Final authorization

The platform is ready for school creation only when the deployment administrator and Platform Super Admin both confirm:

- production security is locked;
 - communication tests are delivered;
 - templates and channel policies are approved;
 - packages and roles are correct;
 - queues, scheduler, and backups are operational;
 - the database contains no staging/demo data.
-